

SESIÓN 3: GESTOR DE TAREAS LOCAL

CON ANGULAR (Standalone + Signals)

Parte 1: GUIADA

El objetivo de esta práctica es construir una aplicación modular de tareas donde el componente Padre (AppComponent) gestiona la lista global, el componente Hijo (TarealtemComponent) muestra cada tarjeta individual de forma independiente, y un Servicio (TareasService) centraliza la lógica mediante Inyección de Dependencias (inject()) y Signals.

PASO 1: Crear el proyecto

Abre tu terminal en la carpeta donde guardes tus proyectos y ejecuta:

```
Bash
ng new gestor-tareas-local --routing=false --style=css
```

Recuerda que cuando pregunte por el formato de hojas de estilo, hay que elegir CSS. Entra en la carpeta del proyecto:

```
Bash
cd gestor-tareas-local
```

Para que la aplicación arranque limpia y sin rastro de la plantilla por defecto:

1. Abre el archivo src/app/app.component.ts.
2. Borra todo su contenido y déjalo con esta estructura vacía pero preparada:

```
TypeScript
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  // Aquí pondremos nuestra lógica más adelante
}
```

3. Abre el archivo `src/app/app.component.html`. Borra todo lo que hay dentro y escribe esto para comprobar que arranca limpio:

```
HTML
<h1>Mi Gestor de Tareas de DAW</h1>
Ejecuta el servidor de desarrollo para probarlo:
Bash
ng serve -o
```

En el navegador ya aparece tu título sin el mensaje de bienvenida por defecto.

PASO 2: Crear el Servicio Centralizado (TareasService)

Para evitar que los datos estén dispersos y aplicar la Inyección de Dependencias, crearemos un servicio global. Ejecuta en el terminal:

```
Bash
ng g s servicios/tareas
```

Esto creará la carpeta `servicios` y el archivo `tareas.service.ts`. Abre el archivo `src/app/servicios/tareas.service.ts` y escribe el siguiente código:

```
TypeScript
import { Injectable, signal } from '@angular/core';

@Injectable({
  providedIn: 'root' // Lo hace global y Singleton automáticamente
})
export class TareasService {
  // Estado centralizado reactivo con Signals
  private _tareas = signal<string[]>([
    'Aprender componentes standalone en Angular',
    'Configurar el flujo de input y output',
    'Dominar la inyección de dependencias con inject()'
  ]);

  // Exponemos las tareas en modo de solo lectura o acceso directo
  readonly tareas = this._tareas.asReadOnly();

  // Método para añadir una nueva tarea
  agregarTarea(nuevaTarea: string) {
    if (nuevaTarea.trim() === '') return;
    this._tareas.update(listaActual => [...listaActual, nuevaTarea]);
  }
}
```

```

}

// Método para eliminar una tarea por su índice
eliminarTarea(index: number) {
  this._tareas.update(listaActual => listaActual.filter((_, i) => i !== index));
}
}

```

PASO 3: Crear el Componente Hijo (TareaItemComponent)

Este componente será el encargado de mostrar cada tarjeta de tarea individual. Ejecuta en el terminal:

```

Bash
ng g c componentes/tarea-item --inline-template --inline-style

```

Esto genera un componente con el HTML y CSS integrados para mayor agilidad. Abre el archivo recién creado `src/app/componentes/tarea-item/tarea-item.component.ts` y sustituye su contenido por este:

```

TypeScript
import { Component, input, output } from '@angular/core';

@Component({
  selector: 'app-tarea-item',
  standalone: true,
  imports: [],
  template: `
    <div class="tarjeta-tarea">
      <span class="texto-tarea">{{ tarea() }}</span>
      <button class="btn-borrar" (click)="onBorrarClicked()">Eliminar</button>
    </div>
  `,
  styles: [`
    .tarjeta-tarea {
      background: #ffffff;
      border: 1px solid #e2e8f0;
      padding: 12px 16px;
      border-radius: 8px;
      margin-bottom: 8px;
      display: flex;
      justify-content: space-between;
      align-items: center;

```

```

    box-shadow: 0 1px 3px rgba(0,0,0,0.05);
  }
  .texto-tarea {
    color: #1e293b;
    font-size: 1rem;
  }
  .btn-borrar {
    background-color: #ef4444;
    color: white;
    border: none;
    padding: 6px 12px;
    border-radius: 6px;
    cursor: pointer;
    font-weight: 500;
    transition: background-color 0.2s;
  }
  .btn-borrar:hover {
    background-color: #dc2626;
  }
  `]
})
export class TarealtemComponent {
  // 1. INPUT: Recibe la tarea obligatoria del Padre
  tarea = input.required<string>();

  // 2. OUTPUT: Emite un evento hacia arriba indicando qué índice se quiere borrar
  onEliminar = output<number>();

  // Recibe el índice del bucle o lo gestiona y emite
  index = input.required<number>();

  onBorrarClicked() {
    this.onEliminar.emit(this.index());
  }
}

```

PASO 4: Unir todo en el Componente Padre (AppComponent)

Ahora vamos a inyectar el servicio en el componente raíz, recoger los datos, mostrar el formulario de alta y listar los componentes hijos utilizando @for. Abre `src/app/app.component.ts` y reemplaza su código por este:

```

TypeScript
import { Component, inject, signal } from '@angular/core';
import { TareasService } from '../servicios/tareas.service';
import { TarealtemComponent } from '../componentes/tarea-item/tarea-
item.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [TarealtemComponent], // <-- IMPORTANTE: Importamos el componente hijo
aquí
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  // Inyección moderna de dependencias
  private tareasService = inject(TareasService);

  // Exponemos las tareas del servicio a la vista
  listaTareas = this.tareasService.tareas;

  // Signal local para el formulario de nueva tarea
  nuevaTareaInput = signal("");

  actualizarTexto(event: Event) {
    const valor = (event.target as HTMLInputElement).value;
    this.nuevaTareaInput.set(valor);
  }

  agregar() {
    this.tareasService.agregarTarea(this.nuevaTareaInput());
    this.nuevaTareaInput.set(""); // Limpiamos el input
  }

  borrar(index: number) {
    this.tareasService.eliminarTarea(index);
  }
}

```

Abre src/app/app.component.html y escribe el diseño de la interfaz:

HTML

```
<main class="container">
  <h1>Gestor de Tareas DAW</h1>
  <p class="subtitulo">Arquitectura Modular con Signals e Inyección de
  Dependencias</p>

  <!-- Formulario de alta -->
  <div class="formulario">
    <input
      type="text"
      [value]="nuevaTareaInput()"
      (input)="actualizarTexto($event)"
      placeholder="Escribe una nueva tarea..."
    />
    <button (click)="agregar()">Añadir Tarea</button>
  </div>

  <!-- Listado de tareas usando el componente hijo -->
  <div class="lista">
    @for (item of listaTareas(); track $index) {
      <app-tarea-item
        [tarea]="item"
        [index]="$index"
        (onEliminar)="borrar($event)"
      />
    } @empty {
      <p class="vacio">No hay tareas pendientes. ¡Buen trabajo!</p>
    }
  </div>
</main>
```

Abre `src/app/app.component.css` para mejorar su aspecto visual:

CSS

```
body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #f1f5f9;
  margin: 0;
  padding: 40px;
}

.container {
  max-width: 600px;
```

```
margin: 0 auto;
background: white;
padding: 30px;
border-radius: 12px;
box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
}
h1 {
color: #0f172a;
margin-bottom: 5px;
}
.subtitulo {
color: #64748b;
margin-top: 0;
margin-bottom: 24px;
}
.formulario {
display: flex;
gap: 10px;
margin-bottom: 24px;
}
input[type="text"] {
flex: 1;
padding: 10px 14px;
border: 1px solid #cbd5e1;
border-radius: 6px;
font-size: 1rem;
}
input[type="text"]:focus {
outline: none;
border-color: #3b82f6;
box-shadow: 0 0 3px rgba(59, 130, 246, 0.1);
}
button {
background-color: #3b82f6;
color: white;
border: none;
padding: 10px 20px;
border-radius: 6px;
cursor: pointer;
font-weight: 600;
transition: background-color 0.2s;
```

```
}  
button:hover {  
  background-color: #2563eb;  
}  
.vacio {  
  text-align: center;  
  color: #94a3b8;  
  font-style: italic;  
}
```

Guarda todos los archivos, vuelve a tu navegador (en <http://localhost:4200>) y comprueba que:

1. Ya no aparece ningún texto por defecto ni errores en la consola.
2. Se cargan las 3 tareas iniciales del servicio.
3. Puedes escribir una nueva tarea en el input, pulsar el botón y ver cómo se añade de forma reactiva.
4. Puedes pulsar el botón "Eliminar" de cualquier tarjeta y comprobar que el componente hijo avisa al padre mediante `output()` para borrarla del servicio global.

Parte 2: Autónoma

El objetivo de esta segunda parte es construir una aplicación web en Angular (v17/v18) aplicando el patrón de arquitectura multicomponente, separación de responsabilidades, Inyección de Dependencias con la función `inject()`, y gestión de estado mediante Signals (con comunicación bidireccional input / output).

PASO 1: Creación del proyecto

1. Abre tu terminal y crea un nuevo proyecto sin enrutamiento y con CSS:

```
Bash  
ng new gestor-tareas-daw --routing=false --style=css
```

2. Entra en la carpeta del proyecto y abre tu editor de código (VS Code):

```
Bash  
cd gestor-tareas-daw  
code .
```

3. Abre `src/app/app.component.html` y borra todo el contenido para que no aparezca la página de bienvenida por defecto (*Hello Angular!*). Pon un título tuyo para comprobar que arranca en blanco.
4. Arranca el servidor de desarrollo:

```
Bash
ng serve --open
```

PASO 2: Creación del Servicio Centralizado (TareasService)

1. Genera un servicio en una carpeta llamada servicios:

```
Bash
ng g s servicios/tareas
```

2. Modifica el archivo generado para que cumpla con los siguientes requisitos:

- Debe estar decorado con `@Injectable({ providedIn: 'root' })` para que sea global (*Singleton*).
- Debe declarar una Signal privada que almacene un array inicial de al menos 3 cadenas de texto (tareas de ejemplo).
- Debe exponer un método público para añadir una nueva tarea a la lista.
- Debe exponer un método público para eliminar una tarea pasándole su índice.

PASO 3: Creación del Componente Hijo (TareaItemComponent)

1. Genera un componente hijo dentro de una carpeta llamada componentes (puedes usar plantillas y estilos para mayor rapidez):

```
Bash
ng g c componentes/tarea-item --inline-template --inline-style
```

2. Configura el componente para que:

- Reciba la tarea del componente padre utilizando la función moderna **input.required<string>()**.
- Muestre la tarea en su plantilla HTML dentro de una tarjeta con diseño limpio.
- Incluya un botón de "Eliminar".
- Emita un evento hacia arriba utilizando la función moderna **output<number>()** al hacer clic en el botón de borrar, enviando el índice de la tarea.

PASO 4: Conexión en el Componente Padre (AppComponent)

1. Inyecta el servicio TareasService en el AppComponent utilizando la función moderna **inject()**.
2. Conecta la lista de tareas del servicio con el componente padre.

3. Prepara una Signal local y un formulario básico (input de texto + botón) en la plantilla del padre para dar de alta nuevas tareas interactivamente.
4. Importa y utiliza el componente hijo <app-tarea-item> dentro del bucle @for en el HTML del padre:
 - Pásale la tarea mediante corchetes [tarea].
 - Escucha el evento de borrado mediante paréntesis (onEliminar).
 - Añade un bloque @empty para mostrar un mensaje elegante cuando no queden tareas pendientes.

Comprueba que tu aplicación funciona correctamente:

- La página arranca limpia sin textos por defecto.
- Se muestran las tareas iniciales del servicio.
- Puedes escribir una nueva tarea, pulsar el botón y ver cómo se actualiza la pantalla al instante de forma reactiva.
- Al pulsar el botón "Eliminar" de cualquier tarjeta, el componente hijo avisa al padre y la tarea desaparece de la lista.